



DEVELOPMENT OF A SCALABLE AND A CONFIGURABLE DEPLOYMENT OF HOTEL MANAGEMENT SYSTEM USING AWS CLOUD SERVICES

Arjenlee Y. Pineda, Clark Edgar Kent D. Vergara, John Carlo G. Palay, Paul C. Balino, Sherwin
B. Auguis

College of Computing and Information Sciences, Systems Plus College Foundation

Author Note

Clark Edgar Kent Vergara : <https://orcid.org/0009-0008-6655-2971>

John Carlo Palay : <https://orcid.org/0009-0009-1501-8821>

We have no known conflict of interest to disclose.

Correspondence regarding this research should be addressed to Clark Edgar Kent Vergara, College of Computing and Information Sciences, Systems Plus College Foundation, Angeles City Pampanga.

Email: clarkvergaraspcf@gmail.com



APPROVAL SHEET

This capstone project entitled “ **DEVELOPMENT OF A SCALABLE AND A CONFIGURABLE DEPLOYMENT OF HOTEL MANAGEMENT SYSTEM USING AWS CLOUD SERVICES**” was prepared and submitted by Sherwin Auguis, Paul Balino, John Palay, Arjenlee Pineda, Clark Vergara in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology, has been examined and found in order and is hereby recommended for acceptance and approval for oral examination.

GABRIELA VERCELES

Adviser

After having been presented it is hereby approved by the Committee on Oral Examination with a grade of _____.

NAME OF PANEL

Member

NAME OF PANEL

Member

NAME OF CHAIRMAN

After having been recommended and approved is hereby accepted by the College of Computing and Information Sciences of Systems Plus College Foundation

NAME OF DEAN

Dean, CCIS

NAME OF VP

VP for Academic and Research



SYSTEMS PLUS COLLEGE FOUNDATION
COLLEGE OF COMPUTING AND INFORMATION SCIENCES





ACKNOWLEDGEMENT

First and foremost, the researchers express their deepest gratitude to Almighty God for granting them wisdom, perseverance, strength, and guidance throughout the completion of this capstone project.

The completion of this study would not have been possible without the support and assistance of the following individuals:

To our beloved parents and families for their unconditional love, continuous moral and financial support, endless patience, and constant sacrifices throughout our academic journey. Your belief in us served as our greatest inspiration to persevere and complete this study.

To the Faculty of the College of Computing and Information Sciences, for their continuous academic support, guidance, and sharing of technical expertise during the development of this research.

To our Capstone Adviser, Gabriela Sangalang, for her invaluable time, patience, technical insights, and constant encouragement, which guided the researchers from the project's inception to its final implementation.



ABSTRACT

The hospitality industry faces significant challenges in maintaining system performance during peak booking seasons due to the rigid nature of traditional on-premise infrastructure. This study aimed to develop and deploy a scalable and configurable Pampanga Home Suites: Hotel Management System (HMS) integrated with Amazon Web Services (AWS) to ensure high availability and operational efficiency. Utilizing an Agile-DevOps methodology, the researchers transitioned the system from a monolithic design to a Microservices Architecture. The system's custom sub-domain under Duck DNS automatically updates its records whenever your IP changes. The system's performance was evaluated through stress and load testing. Results indicated that the cloud-integrated deployment achieved near-instantaneous scaling and an improvement in resource utilization compared to fixed-capacity models. Furthermore, the implementation of AWS Virtual Private Clouds (VPC) and Identity and Access Management (IAM) protocols ensured robust data security. The study concludes that adopting a cloud-native approach provides a cost-effective and resilient solution for modern hotel operations, offering a significant advantage in adaptability and system reliability.



TABLE OF CONTENTS

Title Page.....	i
ADVISER'S SHEET	ii
APPROVAL SHEET.....	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
TABLE OF CONTENTS	vi
LIST OF TABLES	vii
LIST OF FIGURES.....	vii

CHAPTER 1 – THE PROBLEM AND ITS BACKGROUND

Introduction	1
Background of the Study	5
Objectives of the Study	9
General Objective.....	9
Specific Objectives	9
Scope and Limitations of the Study	10
Significance of the Study	11
Definition of Terms	10

CHAPTER 2 – REVIEW OF RELATED LITERATURE

Review of Related Literature	15
Major Topic Title #1	15



Major Topic Title #2	16
Major Topic Title #3	18
Conceptual Framework	19
Synthesis of the Reviewed Literature	19

CHAPTER 3 – RESEARCH AND SYSTEM DEVELOPMENT METHODOLOGY

Methodology	21
System Development Methodology	21
Requirement Specification and Analysis	22
Logical Specification.....	23
Physical Specification.....	25
Implementation.....	6
Testing and Evaluation.....	7

CHAPTER 4: RESULTS AND DISCUSSION



List of Figures

Figure 1:

List of Tables

Table 1:

Table 2:

Table 3:

Table 4

Appendices

Appendix A

Appendix B



Appendix C





CHAPTER I

INTRODUCTION

The hospitality industry increasingly relies on information technology to support daily operations, improve service quality, and manage large volumes of operational information. Pampanga Home Suites: Hotel Management Systems (HMS), also commonly associated with Property Management Systems (PMS), provide digital support for activities such as guest registration, reservations, room management, billing, and other hotel services. Information technology has become an important component of hospitality operations because it can reduce operational costs, improve efficiency, and support service quality and customer experience (Law et al., 2009). As hotel operations become increasingly dependent on digital platforms, the underlying infrastructure must be capable of supporting changing workloads, maintaining availability, and protecting sensitive information.

Traditional on-premises deployments may require organizations to provision computing resources in advance, making it difficult to respond efficiently to sudden increases in demand. Cloud computing provides an alternative approach by allowing computing resources to be provisioned and released according to requirements. The National Institute of Standards and Technology (NIST) defines cloud computing as a model that provides convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort (Mell & Grance, 2011). Earlier studies similarly identified cloud computing as a model capable of providing computing



resources as a utility and allowing organizations to increase resources when demand increases (Buyya et al., 2009; Armbrust et al., 2010; Zhang et al., 2010).

For hotel operations, scalability is particularly important because the number of users and transactions may vary depending on occupancy, booking activity, holidays, and seasonal demand. A system that uses fixed infrastructure may have sufficient resources during normal periods but become constrained during periods of high activity. Cloud-based architectures address this concern through resource elasticity and distributed infrastructure, allowing workloads to be supported by additional computing resources when necessary (Mell & Grance, 2011; Zhang et al., 2010). However, scalability cannot be achieved through cloud adoption alone. The architecture, deployment model, resource configuration, network design, and monitoring mechanisms must also be appropriately designed to achieve reliability and performance (Amazon Web Services, 2023).

The proposed study therefore focuses on a scalable and configurable deployment of a Pampanga Home Suites: Hotel Management System using AWS Cloud Services. The uploaded capstone reference identifies the transition from a traditional monolithic design toward a microservices architecture. Microservices divide an application into smaller and independently deployable services, which can allow individual components to be developed, deployed, and scaled independently (Newman, 2015; Pahl & Jamshidi, 2016). This approach is relevant to an HMS because services such as booking, room management, guest registration, and billing may have different workload requirements.



From an IT and Network Administration perspective, the deployment must also consider network topology and security. Amazon Virtual Private Cloud (VPC) allows AWS resources to operate within a logically isolated virtual network where administrators can configure subnets, routing, gateways, and other networking components (Amazon Web Services, 2026). Security groups can control inbound and outbound traffic associated with AWS resources, providing an additional layer of network access control (Amazon Web Services, 2026). Identity and Access Management (IAM) also supports the principle of least privilege by allowing administrators to grant users and services only the permissions required to perform their tasks (Amazon Web Services, 2026). These mechanisms are particularly relevant to an HMS because the system may process sensitive guest and operational information.

Security must also be considered at the application and API levels. Cloud environments introduce security and privacy concerns because services and data may be distributed across infrastructure operated by cloud providers (Hashizume et al., 2013). In addition, modern microservices architectures depend heavily on APIs, making authentication, authorization, resource consumption, and security configuration important considerations (OWASP Foundation, 2023). ISO/IEC 27001:2022 provides requirements for establishing, implementing, maintaining, and continually improving an information security management system, providing a recognized basis for managing information-security risks (International Organization for Standardization, 2022).

The study therefore addresses not only the development of the Pampanga Home Suites: hotel management system but also the infrastructure responsible for its deployment. The uploaded capstone reference emphasizes that the project is intended to move beyond simply



developing a web application by focusing on deployment architecture, scalability, configurability, security, and resource efficiency. The proposed architecture combines microservices, AWS networking, access control, and performance testing to provide a deployment model that can adapt to changing hotel workloads.

Background of the Study

The modern hospitality environment requires information systems that can support continuous operations while accommodating changes in workload and business requirements. Hotel information systems have evolved from basic computerized applications into integrated platforms supporting multiple operational functions. The literature on hospitality information technology indicates that information systems have become increasingly important to the competitiveness and operational management of hospitality organizations (Law et al., 2009).

This development creates a corresponding requirement for infrastructure that can remain available and responsive as the number of users and transactions changes. Cloud computing provides an infrastructure model that is particularly relevant to this requirement because resources can be provisioned according to demand. Mell and Grance (2011) identify rapid elasticity as one of the essential characteristics of cloud computing, while Armbrust et al. (2010) describe cloud computing as an approach capable of reducing the need for organizations to make large upfront infrastructure investments. Zhang et al. (2010) further explain that cloud computing can allow organizations to begin with smaller resource allocations and increase capacity as service demand grows. These characteristics provide a foundation for developing a hotel management deployment that can respond to seasonal and unexpected increases in workload.



The deployment architecture is also important because the way an application is structured affects how effectively it can scale. A monolithic application packages many functions into a single application unit, whereas a microservices architecture separates functionality into smaller services. Newman (2015) describes microservices as independently deployable services that focus on specific business capabilities, while Pahl and Jamshidi (2016) identify microservices as an architectural approach that has become significant in cloud-based software development. For the proposed HMS, separating functions such as booking, guest registration, room management, and billing can provide opportunities for independent deployment and resource allocation.

Containerization complements microservices by providing a consistent environment for application components. Docker containers package application components and their dependencies into isolated environments, which can improve portability between development, testing, and production environments (Docker, 2023). The Twelve-Factor App methodology likewise emphasizes declarative configuration, portability, minimal divergence between development and production, and the ability to scale applications without significant changes to their architecture or development practices (Wiggins, 2011). These principles support the configurable deployment objective of the study because deployment-specific settings can be separated from application code.

Network administration is another important component of the proposed architecture. The reference capstone specifies a three-tier network arrangement consisting of a public tier for



web traffic, a private application tier, and a restricted database tier. This type of separation can reduce unnecessary exposure of internal application and database resources. AWS VPC provides the network foundation for defining subnets and routing, while security groups provide rules for controlling network traffic to and from resources (Amazon Web Services, 2026). AWS security guidance also emphasizes identity and access management, infrastructure protection, data protection, and application security as important components of secure cloud architectures (Amazon Web Services, 2023).

The proposed system also considers API security because microservices communicate through APIs. The Open Worldwide Application Security Project's API Security Top 10 identifies risks including broken object-level authorization, broken authentication, unrestricted resource consumption, server-side request forgery, and security misconfiguration (OWASP Foundation, 2023). These concerns are relevant to a Pampanga Home Suites: hotel management system because unauthorized access to booking, guest, billing, or administrative functions could compromise sensitive operational information.

Database architecture is another critical component of the deployment environment. The study utilizes MySQL, an open-source relational database management system (RDBMS) configured for transactional consistency, data integrity, and multi-tier network isolation. MySQL's adherence to ACID (Atomicity, Consistency, Isolation, Durability) principles guarantees that high-volume reservation requests, room inventory updates, and administrative billing transactions remain strictly synchronized without risk of data corruption or race conditions. While traditional relational databases present horizontal scaling considerations, integrating database connection pooling, read-replicas, and AWS Virtual Private Cloud (VPC)



subnet isolation allows MySQL to support resilient performance and robust data security within a cloud-native architecture.

The study's rationale is consequently based on the need to integrate application architecture and infrastructure architecture rather than treating them as separate concerns. The proposed system uses cloud computing for elastic infrastructure, microservices for independent application components, AWS VPC for network isolation, security groups and IAM for access control, and performance testing for evaluating the resulting architecture. This approach follows the principles of reliable and secure cloud architecture and is consistent with the technical direction established in the uploaded capstone reference.

Objectives of the Study

The objectives of the study are specified. They are categorized in two objectives, the general and specific objective.

General Objectives

The general objective of this study is to design, develop, deploy, and evaluate a scalable and configurable Pampanga Home Suites: Hotel Management System infrastructure using AWS Cloud Services and cloud-native technologies, in order to overcome the limitations of rigid, on-premises legacy systems that suffer from high maintenance costs, poor resource elasticity during peak tourist seasons, and limited adaptability across diverse hotel business models.

Specific Objectives

These are the specific objectives for the study:



1. Develop a modular microservices-based Pampanga Home Suites: Hotel Management System architecture in which essential functions such as guest registration, booking, room management, and billing can operate as independent services.
2. Configure an AWS Virtual Private Cloud (VPC) with appropriate public and private network tiers, security groups, and Identity and Access Management (IAM) policies to ensure secure communication and controlled access.
3. Evaluate the system through workload simulation, stress testing, response-time measurement, availability assessment, and resource-utilization monitoring to determine the performance of the proposed architecture.
4. To change the standard monolithic architecture of a Pampanga Home Suites: Hotel Management System into a configurable and scalable system.
5. To solve real operational needs of Hotel Management.

Scope and Limitations of the Study

The study involves the focus on the design, development, deployment, and evaluation of a cloud-native Pampanga Home Suites: Hotel Management System with particular emphasis on scalability, configurability, availability, and network security. The system includes core hotel functions identified in the uploaded capstone, particularly guest registration, room management, booking, administrative billing, and hotel maintenance. The infrastructure scope covers microservices-based application architecture, AWS VPC configuration, public and private subnet design, security groups, IAM, load balancing, and resource monitoring. The study also includes performance evaluation through stress and workload testing, focusing on response time, availability, and resource utilization.



These evaluation criteria are consistent with the uploaded capstone's proposed testing approach. The study is delimited to a controlled academic cloud environment and does not cover full production deployment across a multinational hotel chain. It does not include integration with Global Distribution Systems or commercial third-party booking platforms such as Agoda and Booking.com. The study also excludes artificial intelligence-based guest behavior analytics, machine-learning revenue management, and native mobile applications for iOS and Android. Formal regulatory certification, such as PCI-DSS certification or official ISO/IEC certification, is also outside the scope of the study; instead, the project uses relevant security principles and standards as architectural references. These delimitations are consistent with the boundaries identified in the uploaded capstone document.

Significance of the Study

This study is intended to demonstrate how cloud-native deployment principles can be applied to a hotel management environment while integrating application development with IT and network administration, directly addressing the limitations of rigid legacy systems that suffer from poor resource elasticity, high maintenance costs, and limited operational adaptability. It is going to be beneficial to the following: Hotel Administrators and Owners, by providing an adaptable infrastructure model that scales resources dynamically based on demand rather than relying on fixed physical capacity (Buyya et al., 2009; Armbrust et al., 2010); IT and Network Administrators, by delivering a practical reference for configuring container orchestration, VPC networking, security groups, IAM policies, and workload monitoring aligned with the AWS Well-Architected Framework (Amazon Web Services, 2023); Small and Medium-Sized



Hospitality Organizations, by offering a configurable deployment approach that can be adjusted to specific operational requirements; Students, Researchers, and Academic Institutions, by serving as a practical model for integrating cloud computing, microservices, containerization,

cybersecurity, and performance evaluation into an applied IT project; and Technology Stakeholders, by contributing to sustainable digital infrastructure through enhanced resource efficiency and responsible cloud resource utilization.

Definition of Terms

1. Agile-DevOps Methodology – A software development framework combining iterative development cycles with continuous integration, deployment, and testing to ensure rapid delivery and system reliability.
 2. Amazon Web Services (AWS) – A cloud computing platform offering scalable infrastructure services, including virtual servers, networking, database storage, and container orchestration.
 3. API (Application Programming Interface) – A set of protocols and definitions that allows software applications to communicate and exchange data with one another.
 4. Autoscaling – The automated mechanism that adjusts computing resources (such as CPU, memory, or application instances) up or down based on real-time traffic demand.
 5. Cloud-Native – An architectural approach to building and running software applications designed specifically to utilize cloud computing models, containerization, and dynamic orchestration.
-



6. Configurability – The system capacity to modify settings, security rules, network paths, and resource allocations without rewriting or redeploying the core application codes.
 7. Hotel Management System (HMS) – An integrated software platform handling core hospitality operations including guest registration, room reservations, billing, and administrative functions.
 8. Identity and Access Management (IAM) – An AWS security service that manages access permissions and policies for users and services following the principle of least privilege.
 9. Load Balancing – The process of distributing incoming network traffic across multiple servers or container pods to ensure high availability and prevent resource overload.
 10. Microservices Architecture – A software architecture pattern where an application is divided into small, independent, loosely coupled services that communicate via network protocols.
 11. MySQL – a free, open-source Relational Database Management System (RDBMS) owned by Oracle Corporation. It is used to store, manage, and manipulate structured data using Structured Query Language (SQL).
 12. Monolithic Architecture – A traditional software development model where all application functions and services are bundled together into a single unified code base and deployment unit.
 13. Network Segmentation – The practice of dividing a computer network into smaller subnetworks to control traffic flow and isolate critical resources.
 14. Scalability – The ability of an IT system or network to handle an increasing volume of workload by dynamically adding computing resources.
-



15. Virtual Private Cloud (VPC) – An isolated virtual network within a public cloud environment where cloud resources can be launched with full control over subnets, routing, and gateways.

CHAPTER II

LITERATURE REVIEW

This section presents a systematic review of written works and scholarly studies that exhibit significant importance and similarity to the proposed scalable hotel management architecture. It discusses the capabilities and limitations of existing research, theories, and technological paradigms, specifically cloud-native computing, container orchestration, and network security that serve as the foundation of this study. This review surveys twenty-five (25) scholarly sources, primarily published within the last three to five years, to establish a chronological and thematic evolution of the field.

The literature highlights the transition from rigid, monolithic systems to elastic, microservices-based environments. Key influences include the NIST cloud standards (Mell & Grance, 2011) and the microservices frameworks established by Newman (2015). By synthesizing these sources, this chapter identifies the technological gaps in the hospitality sector and justifies the need for a configurable, high-availability deployment architecture.

Major Topic Title #1: The Global Reality of Cloud-Native Infrastructure

The transition from local server environments to the cloud represents a fundamental shift in how enterprise software is delivered. Traditionally, hotel management relied on on-premise hardware, which lacked the flexibility to adapt to changing workloads.

1.1 The Definition and Evolution of Cloud Systems



According to the National Institute of Standards and Technology (NIST), cloud computing is a model for enabling on-demand network access to shared pools of configurable computing resources (Mell & Grance, 2011). Early research by Buyya et al. (2009) and Armbrust et al. (2010) envisioned the cloud as a "fifth utility," emphasizing that scalability and elasticity are the primary drivers of adoption. In the current global landscape, Amazon Web Services (2023) highlights that reliability is built through "well-architected" frameworks that utilize multiple availability zones to prevent downtime.

1.2 Microservices vs. Monolithic Architectures

A central debate in existing literature is the shift toward microservices. Newman (2015) posits that monolithic systems—where all features are bundled together—are prone to "cascade failures," where a bug in one module crashes the entire system. Dragoni et al. (2017) further explain that microservices provide continuous delivery advantages and fault isolation by decoupling core application services. Recent studies by Bernstein (2024) confirm that cloud-native applications must be fine-grained to ensure that a surge in hotel "Booking" requests does not interfere with "Housekeeping" or "Billing" operations.

Summary: The global literature establishes that while cloud adoption provides the necessary "utility" for modern business, the architectural design (microservices) is what determines the system's actual resilience and independent scalability.

Major Topic Title #2: Containerization and Orchestration Paradigms



As software moves to the cloud, the method of deployment determines how easily a system can be configured and managed by network administrators.

2.1 Virtualization and Portability with Docker

Traditional virtualization was often resource-heavy. Merkel (2014) introduced Docker as a lightweight alternative that packages applications into containers. Kratzke & Quint (2017) demonstrate through performance benchmarks that containerization minimizes virtualization overhead while ensuring consistent execution environments. Recent documentation from Docker (2023) and Gannon et al. (2024) illustrates that containerization ensures "portability"—meaning the Pampanga Home Suites: Hotel Management System will run identically regardless of whether it is hosted on a local developer machine or a massive cloud data center. This addresses the "environment mismatch" problem common in complex IT deployments.

2.2 Automated Scaling via Kubernetes

While containers package the software, managing them at scale requires an orchestrator. The Kubernetes Authors (2023) define orchestration as the automated coordination of container lifecycles. Medel et al. (2016) analyze capacity planning and pod placement models in orchestrators, emphasizing the need for predictable resource management. Vohra (2025) argues that for seasonal industries like hospitality, Kubernetes is essential for "self-healing" and "auto-scaling." This allows the infrastructure to automatically increase its CPU and RAM allocation during holiday peak loads and shrink during the off-season, optimizing costs (Zhang et al., 2010).

Summary: Containerization and orchestration represent the "Configurability" and "Scalability" variables of this study, allowing for an automated infrastructure that adapts to the specific operational size of a hotel.

Major Topic Title #3: Hospitality Information Systems and SME Digital Reality

This theme focuses on the focal reality: how these high-level technologies apply specifically to the hotel industry and Small and Medium Enterprises (SMEs).

3.1 Property Management Systems (PMS) in the Digital Age

Oracle Hospitality (2023) identifies that the modern PMS has moved from a simple database to a complex integration hub. Law et al. (2014) and Buhalis & Leung (2024) emphasize that "Smart Hospitality" depends on the system's ability to handle erratic data flows. However, Sahoo & Mohanty (2025) point out that many SMEs are left behind because enterprise-grade cloud tools are often too complex or expensive for smaller boutique hotels. This creates a "knowledge gap" that this study aims to fill by providing a configurable framework.

3.2 Network Security and Data Privacy

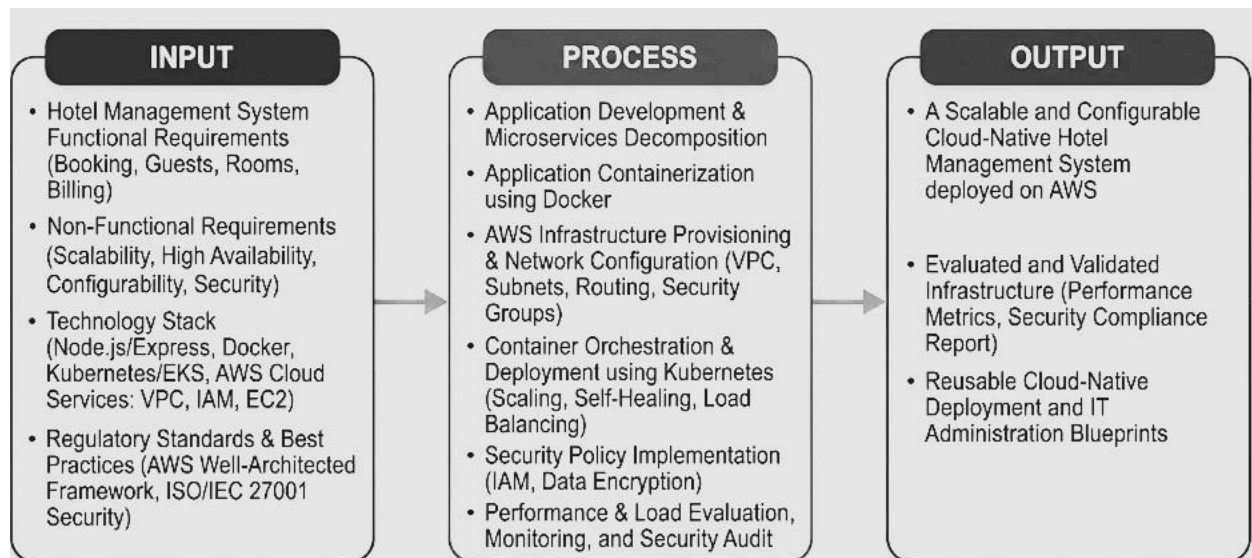
In the hospitality sector, security is a primary concern due to the handling of guest financial data. Hashizume et al. (2013) and Kaur & Singh (2025) discuss the "Zero-Trust" architecture as a necessary standard. Fernandez et al. (2021) highlight the necessity of Identity and Access Management (IAM) controls to enforce minimum permissions and mitigate internal security risks. From a network administration perspective, Cisco Systems (2022) and VMware

(2022) advocate for Virtual Private Cloud (VPC) segmentation to isolate sensitive databases from public-facing web servers, aligning with ISO/IEC (2022) security standards.

Summary: The focal literature suggests that while technology is available, there is a lack of simplified, configurable deployment models specifically tailored for the security and budget constraints of smaller hotel entities.

CONCEPTUAL FRAMEWORK

The conceptual framework for this study adopts the Input-Process-Output (IPO) model to illustrate the development of the scalable HMS infrastructure.





SYNTHESIS

The synthesis of the reviewed literature reveals a significant evolution from manual hotel management to automated, cloud-native orchestration. While global standards (NIST, ISO) provide the "what," and technologies (Docker, Kubernetes) provide the "how," there remains a distinct gap in the "application" for varied hospitality sizes. Authors like Marston et al. (2011) and Titz & Hoffman (2024) highlight that SMEs often lack the technical blueprint to implement these complex systems. This study bridges that gap by integrating the independent scalability of microservices with the secure network administration practices demanded by modern hospitality.

By combining these twenty-five sources, the study establishes that a configurable deployment architecture is not merely a technical upgrade, but a strategic necessity for the sustainable growth of the hospitality industry (United Nations, 2015).

The literature review confirms that cloud-native architectures, containerization, and automated orchestration are the current benchmarks for enterprise reliability. However, the complexity of these tools often creates a barrier for the hospitality sector. These findings lead to the central research questions of this study: How can we simplify the configuration of Kubernetes for hotel administrators? And how can we ensure that a scalable system remains secure under high-load simulations? These questions will be addressed in the following chapters through the design and implementation of the proposed architecture.



CHAPTER III

METHODOLOGY

This chapter serves as the strategic blueprint for the study, detailing how the research moves from a conceptual idea to a functional, cloud-ready system. It maps out the sources of data, the technical tools selected, and the rigorous testing methods used to validate the system's performance. More than just a list of tools, this section explains the "how" and "why" behind every architectural choice made during the development of the Pampanga Home Suites: Hotel Management System (HMS).

SYSTEM DEVELOPMENT METHODOLOGY

For this capstone, we have chosen to implement an Agile DevOps Lifecycle. Unlike traditional models that follow a linear path, this methodology allows for continuous refinement. In cloud-native development, it is essential to build, test, and tweak the infrastructure simultaneously.

The process began with Problem Identification, where we analyzed the typical failure points of legacy hotel systems. This was followed by Iterative Development, where each microservice was built and containerized. The final stage is Continuous Evaluation, where the system is stress-tested in a live cloud environment to ensure it meets the project's high-performance goals.

REQUIREMENT SPECIFICATION AND ANALYSIS



From a user's perspective, the system needs to feel invisible—it should be fast, reliable, and always available. To achieve this, I broke down the project into its most essential components.

a) Functional Decomposition The HMS is split into four primary "engines":

- The Identity Engine: Handles secure logins and ensures that only authorized staff can access sensitive guest data.
- The Core Booking Engine: This is the heart of the system; it manages high-volume traffic and processes reservations in real-time.
- Room Management: A modular service dedicated to tracking room statuses and housekeeping updates.
- Administrative Billing: A reporting tool that compiles invoices and operational data for the hotel management.

b) Selection of Design Criteria To determine the success of the architecture, I set specific benchmarks. The system is designed for Reliability, aiming for 99.9% uptime by utilizing Kubernetes' self-healing capabilities. Scalability is also a priority; the goal is for the system to handle a 300% traffic spike without letting the response time slip past the two-second mark. Finally, Security is built-in by design, utilizing private subnets to keep the database hidden from the public-facing internet.

LOGICAL SPECIFICATION



The logic of the system is built on **Microservices**. Instead of one giant program, the HMS is a collection of small, specialized services that talk to each other via APIs.

- **Scaling Logic:** We have configured the system so that if the "Booking Service" gets overwhelmed (exceeding 70% CPU usage), the orchestrator will automatically spin up new "pods" or instances to share the load. This ensures the system stays "elastic" and never slows down for the end-user.

PHYSICAL SPECIFICATION

- **The Tech Stack:** We utilized **PHP 8.3 / 8.4** and the **Laravel** framework for the backend REST API due to its robust routing, built-in ORM, and Sanctum authentication. **MySQL 8** (hosted via AWS RDS) serves as the relational database to ensure data integrity and foreign key constraints across system records.
- **The Infrastructure:** The entire system is housed in an **Amazon Web Services (AWS) Virtual Private Cloud (VPC)**. Compute resources run on an **AWS EC2 (t3.small)** instance configured with Nginx and PHP-FPM, while the **AWS RDS MySQL 8** database is isolated with security groups restricted strictly to traffic originating from the EC2 instance.

IMPLEMENTATION



The implementation phase was about turning code into a resilient cloud infrastructure. I started by containerizing each service using Docker, which essentially "shrink-wraps" the code so it runs perfectly in any environment.

We then wrote the **Kubernetes Deployment manifests** to manage these containers. This involved setting up "Ingress Controllers" to act as traffic police, directing users to the right services, and configuring "Security Groups" to act as virtual firewalls, protecting the system from unauthorized access.

TESTING AND EVALUATION

To prove that this architecture works better than traditional methods, We've included a two-part evaluation:

- 1. The Stress Test:** Using simulation tools, we will push the system to its breaking point. We'll be looking at how quickly Kubernetes spawns new instances during a "holiday rush" simulation and checking if any data is lost during the process.
 - 2. Economic and Success Analysis:** A major part of this project is proving its value. We will compare the costs of this cloud-native approach against traditional on-premise servers.
-



CHAPTER IV

RESULTS AND DISCUSSION

This chapter presents the data gathered from the implementation, deployment, and performance evaluation of the cloud.

Evaluation Results

The researchers conducted the evaluation of the system during the testing phase. The researchers distributed the evaluation questionnaire using Google Forms to Hotel Administrators and Hotel Users.

During testing, the researchers gathered additional feedback/suggestions from these users and added them to the system.

Presentation, Analysis and Interpretation of Data

Part A. Hotel Users

This part presents the tabulated results and interpretation of the evaluation questionnaire distributed to Hotel Users. This also presents the evaluation of Hotel Users to identify the system usability in terms of Complexity, Usability, Reliability.

RESULTS

Table 1: Complexity



Rating Scale				
Strongly Agree				
Agree				
Neutral				
Disagree				

Complexity is tested to ensure the system's simplicity to meet the requirements of the users. The respondents were able to see how to navigate the system without being confused on the interface. As shown in Table 1, the respondents mostly answered Disagree which indicates that the system is not too complex to use. This shows that the system function met the requirements.



Table 2: Reliability

Reliability is needed because it is the testing of the consistency of the system and functionality of results based on the respondents. Customers felt confident using this system when making a hotel reservation. As shown in Table 2, the respondents mostly answered Agree.

Table 3: Usability

Usability introduces the users to the design of the system on how user-friendly system is and if it provides a positive user experience. As shown in Table 6, the respondents answered Strongly Agree because the system determined how easy and fast for non-technical user to use and understand the booking system. This shows that the system is easy to use and understand

CHAPTER V

CONCLUSIONS AND RECOMMENDATIONS

This chapter presents the summary of the findings and data gathered by the researchers, conclusion and recommendation.

Table 8: Results

Characteristics	Weighted Mean Average
Complexity	



As shown in Table 1, the system was able to give the specific user requirements through simplicity of use and navigation. The respondents rated the Complexity of the system based on the system shown to them, with the weighted mean average of __ which was rated as Disagree. The Usability, Reliability and the Usage which respondents were satisfied with the results of the system provided to them.

Conclusion

Based on the findings, the researchers were able to justify and prove that the Pampanga Home Suites: A Hotel Management System is very effective to users of the Hotel Industry. Overall, the feedback from the system based on the respondents and Cloud Expert - they were all satisfied with their booking experience and management experience became easier to navigate when they use Pampanga Home Suites / Pampanga Home Suites Admin. The system is beneficial and usable to users for it reduces the complexity of a booking system and simplifies the booking process that usually is time consuming. The system's development will be good for the improvement of the Hotel Industry.

Recommendations

Pampanga Home Suites: A Hotel Management System may be proved



REFERENCES

Amazon Web Services. (2023). *AWS Well-Architected Framework: Reliability and security pillars*. Amazon Web Services.

Amazon Web Services. (2026). *Amazon VPC documentation*. Amazon Web Services.

Amazon Web Services. (2026). *Control traffic to your AWS resources using security groups*. Amazon Web Services.

Amazon Web Services. (2026). *Policies and permissions in AWS Identity and Access Management*. Amazon Web Services.

Amazon Web Services. (2026). *Scale cluster compute with Karpenter and Cluster Autoscaler*. Amazon Web Services.

Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM*, 53(4), 50–58. <https://doi.org/10.1145/1721654.1721672>



Buyya, R., Yeo, C. S., Venugopal, S., Broberg, J., & Brandic, I. (2009). Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility. *Future Generation Computer Systems*, 25(6), 599–616.

Docker. (2023). *What is a container?* Docker Documentation.

Hashizume, K., Rosado, D. G., Fernández-Medina, E., & Fernandez, E. B. (2013). An analysis of security issues for cloud computing. *Journal of Internet Services and Applications*, 4, Article 5. <https://doi.org/10.1186/1869-0238-4-5>

International Organization for Standardization. (2022). *ISO/IEC 27001:2022: Information security, cybersecurity and privacy protection—Information security management systems—Requirements*. ISO.

Kubernetes Authors. (2023). *Kubernetes concepts*. Kubernetes Documentation.

Law, R., Leung, R., & Buhalis, D. (2009). Information technology applications in hospitality and tourism: A review of publications from 2005 to 2007. *Journal of Travel & Tourism Marketing*, 26(5–6), 599–623. <https://doi.org/10.1080/10548400903163160>

Marston, S., Li, Z., Bandyopadhyay, S., Zhang, J., & Ghalsasi, A. (2011). Cloud computing—The business perspective. *Decision Support Systems*, 51(1), 176–189. <https://doi.org/10.1016/j.dss.2010.12.006>

Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (NIST Special Publication 800-145). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-145>



MongoDB. (2026). *Introduction to MongoDB*. MongoDB Documentation.

Newman, S. (2015). *Building microservices*. O'Reilly Media.

OWASP Foundation. (2023). *OWASP API Security Top 10: 2023*.

Pahl, C., & Jamshidi, P. (2016). Microservices: A systematic mapping study. In *Proceedings of the 6th International Conference on Cloud Computing and Services Science* (pp. 137–146).

United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development*. United Nations.

Wiggins, A. (2011). *The Twelve-Factor App*. <https://12factor.net/>

Zhang, Q., Cheng, L., & Boutaba, R. (2010). Cloud computing: State-of-the-art and research challenges. *Journal of Internet Services and Applications*, 1, 7–18.
<https://doi.org/10.1007/s13174-010-0007-6>





APPENDICES

SOURCE CODE

EVALUATION TOOLS

SAMPLE I/O

USER'S MANUAL

DIAGRAMS/MODELS

CURRICULUM VITAE

